

CSE 60745 – Fall 2026

Graph Learning: Techn & Apps

Instructor: Prof. Fanny Ye (yve7@nd.edu)

TA: Yijun Ma (yuma7@nd.edu)

2026.09.23

Outline

- ❖ **HW1 Solution**
- ❖ **Practice I: Homogeneous Graph Embedding**
- ❖ **Practice II: Heterogeneous Graph Embedding**

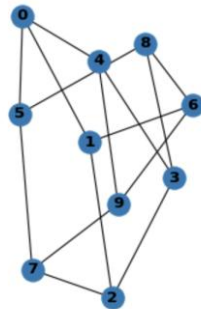
HW-1 Solution

HW1 Solution

Part A: Graph Creation and Modification

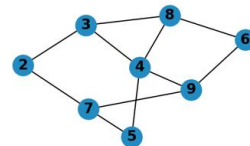
- Graph Creation

```
import networkx as nx
G = nx.Graph()           # Create Graph
G.add_edges_from([(0,1),(0,4),...]) # Add Edges
nx.draw(G, with_labels=True, font_weight='bold') # Visualization
```



- Graph modification & Connected Components

```
G.remove_node(1)        # Remove Node
G.remove_edge((0,4))     # Remove Edge
nx.connected_components(G) # List of Connected Components
```



HW1 Solution

Part B: Graph Basics

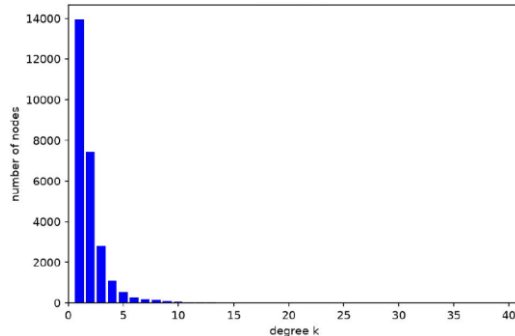
- Graph Creation

```
G = nx.Graph()
len(G) # Node Number
G.number_of_nodes() # Node Number
G.number_of_edges() # Edge Number
nx.number_connected_components(G) # Connected Component Number
```

- Graph Essentials

```
GCC_list = sorted(nx.connected_components(G), key=len, reverse=True)
G_sub = G.subgraph(GCC_list[0]) # Largest Connected Component
G_sub = G.subgraph(GCC_list[1]) # Second Largest Connected Component
```

```
# Returns a list, where the k-th element indicates the number of nodes with degree k
degree_hist = nx.degree_histogram(G)
```



HW1 Solution

Part B: Graph Basics

- Graph Measures

```
nx.eigenvector_centrality(G)      # Eigenvector Centrality
nx.katz_centrality(G)            # Katz Centrality
nx.average_clustering(G)          # Average Clustering Coefficient
nx.jaccard_coefficient(G, G.edges()) # Jaccard similarity over connected node pairs
```

- Note:** If you use `nx.jaccard_coefficient(G)` for Jaccard similarity computation, networkx will compute the similarity of all non-edge node pairs.

HW2-Part(B): 30 pts

Practice I:

Graph Embedding Over Homogeneous Structures

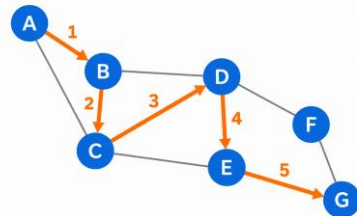
Hands-on: Graph Embedding Over Homogeneous Structures

Recap: Random Walk-Based Graph Embedding

- DeepWalk: Uniform random walk sampling + Skip-gram contextualized embedding
- Node2vec: Biased random walk sampling + Skip-gram contextualized embedding
- Struct2vec: Structural similarity modeling + Multi-layer random walk

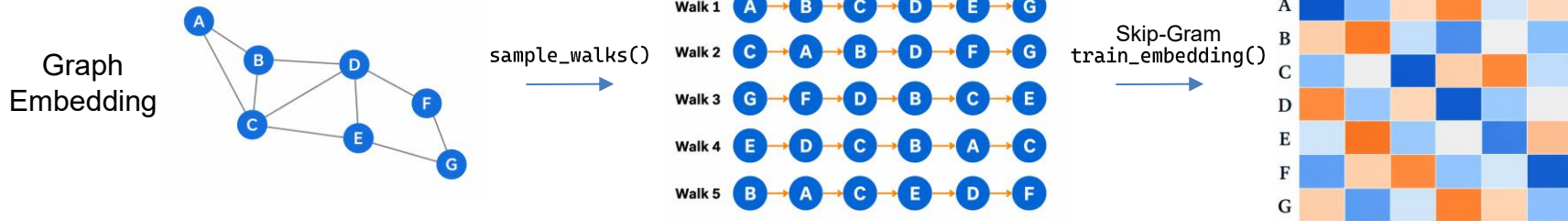
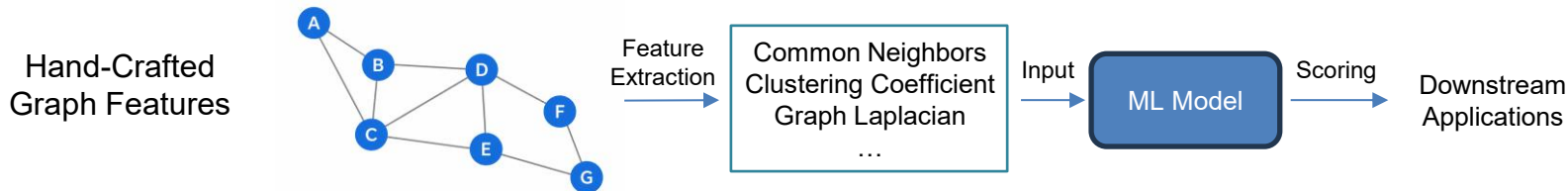
Apply Homogeneous Graph Embeddings to Social Network Analysis

- Two downstream tasks with a single set of user embeddings
 - Friend recommendation: Given a set of candidate users, real friends should be ranked top.
 - Multi-label user node classification: Predict the country label of given user.
- Implement existing graph embedding techniques or design your own embedding methods
 - **Dataset & Code:** https://drive.google.com/file/d/1AtUm_wJboF4t-hxON8r01eYzdhPX9Bnt/view?usp=sharing
 - It is recommended to first re-implement DeepWalk/Node2Vec, and then test your own ideas for random walk sampling strategy and other components.

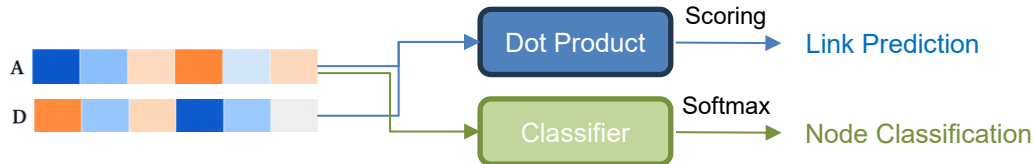


Hands-on: Graph Embedding Over Homogeneous Structures

Demo: How to apply graph embeddings to downstream tasks



Embedding-Based
Downstream
Prediction for A



(Optional)

Practice II:

Heterogeneous Graph Embedding



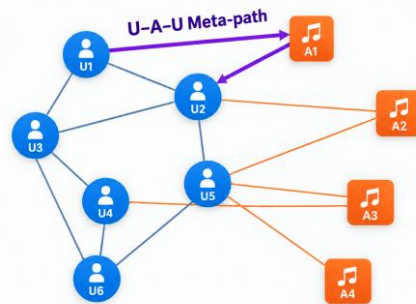
UNIVERSITY OF
NOTRE DAME

| COLLEGE OF ENGINEERING

Hands-on Practice: Heterogeneous Graph Embedding

Recap: Heterogeneous Graph Embedding

- Heterogeneous Graph: Graphs with multiple types of nodes and edges.
- Meta-Path: A sequence template of node/edge types with certain semantic meaning.
- Metapath2vec: Meta-path bounded random walk sampling + Skip-gram contextualized embedding



Heterogeneous Social Network Analysis

- Given new “artist” nodes in LastFM-Asia, enrich user embeddings with heterogeneous graph structures for the same downstream tasks in Practice
- Implement existing graph embedding techniques or design your own embedding methods
 - **Please follow the instructions for Task 4 in the same notebook.**
 - It is recommended to first re-implement metapath2vec, test different meta-path combinations, and then test your own ideas for random walk sampling strategy and other components.

CSE 60745 – Fall 2026

Thank You

Questions: yma7@nd.edu